```python
import random

ANZAHL_SPIELE = 100000

KOMBINATIONEN = [
    "Royal Flush",
    "Straight Flush",
    "Poker",
    "Full House",
    "Flush",
    "Strasse",
    "Drilling",
    "Zwei Paare",
    "Ein Paar",
    "High Card"
]


# Zieht fünf unterschiedliche Karten
def zieheKarten():
    return random.sample(range(52), 5)



def erstelleKartenDaten(hand):
    werte = []
    farben = []

    for karte in hand:
        # wenn zB 11 % 13 = 11 (rest) also ist wert 13
        werte.append(karte % 13 + 2)
        # wenn man < 12 / 13 immer 0, über 12 = 1 usw. jede Zahl eine Farbe
        farben.append(karte // 13)

    return werte, farben


# Zählt die Häufigkeit der Kartenwerte
def zaehleWerte(werte):
    anzahl = []

    # bei zB [7, 7, 2, 2, 2] -> [7, 2] dass es nicht doppelt gezählt wird
    for wert in set(werte):
        anzahl.append(werte.count(wert))
    # sortieren  also [2, 3]
    return sorted(anzahl, reverse=True)


# Prüft, ob alle Karten dieselbe Farbe haben
def istFlush(farben):
    # set entfernt doppelte, wenn alle gleiche Farbe haben
    return len(set(farben)) == 1


# Prüft, ob die Karten eine Strasse bilden
def istStrasse(werte):
    sortierteWerte = sorted(werte)

    # wenn nt alle verschieden sind kanns keine Straße sein
    if len(set(werte)) != 5:
        return False
```

```
    if sortierteWerte == [2, 3, 4, 5, 14]:
        return True
    # oder wenn erster - letzter Wert Differenz 4 ist
    return sortierteWerte[4] - sortierteWerte[0] == 4


# Prüft auf Royal Flush
def istRoyalFlush(werte, farben):
    return (
        istFlush(farben)
        and sorted(werte) == [10, 11, 12, 13, 14]
    )


# Prüft auf Straight Flush
def istStraightFlush(werte, farben):
    return (
        istFlush(farben)
        and istStrasse(werte)
        and not istRoyalFlush(werte, farben)
    )


# Prüft auf Poker (vier gleiche Karten)
def istPoker(anzahl):
    return anzahl == [4, 1]


# Prüft auf Full House
def istFullHouse(anzahl):
    return anzahl == [3, 2]


# Prüft auf Drilling
def istDrilling(anzahl):
    return anzahl == [3, 1, 1]


# Prüft auf zwei Paare
def istZweiPaare(anzahl):
    return anzahl == [2, 2, 1]


# Prüft auf ein Paar
def istEinPaar(anzahl):
    return anzahl == [2, 1, 1, 1]


# Bestimmt die stärkste Pokerkombination
def pruefeKombination(hand):
    werte, farben = erstelleKartenDaten(hand)
    anzahl = zaehleWerte(werte)

    if istRoyalFlush(werte, farben):
        return "Royal Flush"

    if istStraightFlush(werte, farben):
        return "Straight Flush"

    if istPoker(anzahl):
        return "Poker"

    if istFullHouse(anzahl):
```

```
        return "Full House"

    if istFlush(farben):
        return "Flush"

    if istStrasse(werte):
        return "Strasse"

    if istDrilling(anzahl):
        return "Drilling"

    if istZweiPaare(anzahl):
        return "Zwei Paare"

    if istEinPaar(anzahl):
        return "Ein Paar"

    return "High Card"


# Führt die Simulation durch
def simulierePoker():
    ergebnisse = {}

    for kombination in KOMBINATIONEN:
        # leeres dictionary, jede kombination bis jz 0x
        ergebnisse[kombination] = 0

    for spiel in range(ANZAHL_SPIELE):
        # karte ziehen
        hand = zieheKarten()
        #kombi speichern
        kombination = pruefeKombination(hand)

        ergebnisse[kombination] += 1

    return ergebnisse


# dictionary wird übergeben
def zeigeErgebnisse(ergebnisse):
    print("POKER-SIMULATION")
    print("================")
    print("Anzahl Spiele:", ANZAHL_SPIELE)
    print()

    print(
        f"{'Kombination':<18}"
        f"{'Anzahl':>10}"
        f"{'Prozent':>12}"
    )
    # trennlinie
    print("-" * 40)

    # kombination zeile für zeile ausgeben
    for kombination in KOMBINATIONEN:
        anzahl = ergebnisse[kombination]

        prozent = anzahl / ANZAHL_SPIELE * 100

        print(
            f"{kombination:<18}"
            f"{anzahl:>10}"
```

```
            f"{prozent:>11.5f} %"
        )


# Hauptprogramm
def main():
    ergebnisse = simuliere Poker()
    zeigeErgebnisse(ergebnisse)

# startet dann programm
main()
```